Born, Not Trained: Zero-Training Programs Out-Explore Reinforcement Learning for Game Testing at Equal Budget

Anonymous Author(s)
Affiliation
Address
email

Abstract

Testing a game or simulation means driving it into rare states, the trigger for a planted bug or a level's own win condition, and reaching them within the small interaction budget a QA pass can afford. The standard approach trains a reinforcement-learning explorer for each game, which is expensive, needs per-game tuning, and rarely transfers. We propose a different route. Rather than train a tester, an autonomous LLM-evolution loop writes one as a short program. The pipeline is an iterative evolutionary search, not a single prompt: over many generations, a proposal LLM writes and mutates candidate programs, and a multi-engine fitness keeps the ones that reach the most target states at a fixed budget, re-verifying every reported hit from a clean reset. Each candidate is a compact procedure over a save/restore archive of simulator states, saving promising states and returning to them to explore further. The winning program needs no training: it explores through an adaptive novelty signal and a small bandit over exploration moves that emerged from the search, and it is written once and reused. A single fresh-per-task prompt does not suffice; the strength comes from evolving one program against a diverse multi-engine fitness and transferring it. At an equal interaction budget, this single written program is the best method on four sparse-reward game-testing tasks spanning three engines, reaching on average 91% of the targets against 54% for tuned PPO, and less for every other baseline. It matches what trained RL needs roughly $16\times$ more compute to reach, and it transfers to engines the search never saw. However, the advantage is bounded. On saturated or reward-dense tasks trained RL competes and eventually wins, so writing and training a tester are complementary regimes. A benchmark of planted physics defects grounds every number, and the same replay verifier caught the loop inflating its own score by exploiting a race in its evaluation harness, a self-check we expect autonomous research will increasingly need.

Keywords: autonomous ML research; LLM-driven program synthesis; evolutionary search; game testing; exploration; reinforcement learning; execution-grounded verification

1 Autonomous research result

This Part presents the result the loop produced: the setup and hypothesis, the evidence, and the primary finding. It was drafted by the loop and verified by the human author (Section 2.5).

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Table 1: Where each family sits. Our tester is the one that is budget-matched, training-free, and target-reaching at once; universality across all tasks is *not* claimed (Section 1.3).

	no per-env training	fixed interaction budget	sparse-reward target-reaching	replay-verified
PPO + intrinsic [Tang et al., 2017, Burda et al., 2019, Pathak et al., 2017]	✗	✗	partial	✗
Go-Explore [Ecoffet et al., 2021]	✓	partial	✓	✗
Action-repeat [Machado et al., 2018]	✓	✓	✗	✗
Eureka / PromptPO [Ma et al., 2024, Hatgis-Kessell and Brunskill, 2026]	✗/✓	✗	✗	✗
Written program (ours)	✓	✓	✓	✓

1.1 Introduction

Automated game testing needs an *exploration* policy that drives a game into the rare states that trigger a planted defect or satisfy a level's win condition, and reaches them within the small interaction budget a QA pass can afford. The reflex is to **train** a reinforcement-learning (RL) explorer, usually PPO with an intrinsic-motivation bonus, once per game. But trained RL is costly in interaction, needs per-game reward shaping and tuning, and transfers poorly, so at a tester's real budget it under-explores.

We take the opposite route. Rather than **train** a policy, an autonomous LLM-evolution loop **writes** an exploration *program* and evolves it: over many generations a proposal LLM writes and mutates candidate procedures, and a multi-engine fitness keeps the ones that reach the most target states at a fixed budget, with every counted hit replayed from a clean reset [Romera-Paredes et al., 2024]. Our central finding is a sharp, budget-matched boundary: **at an equal interaction budget, on sparse-reward target-reaching, a single zero-training program the loop wrote out-reaches a tuned RL explorer and every baseline; given 4–$16\times$ more interaction, or on reward-dense tasks, training catches up.** Writing and training the tester are not rivals but regimes, split by reward density and budget, and the loop maps that split cheaply. To our knowledge this is the first budget-matched demonstration that a written exploration program out-reaches trained RL for game testing, with every counted result execution-verified.

Contributions. (i) A loop that discovers game-testing exploration *programs*, and the map of where a written program beats a trained one. (ii) The headline (Fig. 1): on four sparse-reward tasks across three engines, our tester is the single best method at equal budget, above random, action-repeat, plain Go-Explore, and tuned PPO. The win is the discovered strategy, not the archive, and it transfers to engines the search never saw. (iii) A verification backbone, replay-from-reset of every counted hit over a frozen hashed protocol with sealed splits and a reachability calibration, which caught the loop inflating its own fitness (Section 1.4, Section 2.5). The programs, the benchmark, and this Part were produced by the loop. We do not overclaim universality: our tester ties two saturated tasks a trivial baseline also wins, cedes reward-dense Craftax to PPO, and does not yet run at Kinetix protocol scale. We draw that map in full.

1.2 Related work

RL exploration / PPO. Count-based [Tang et al., 2017], RND [Burda et al., 2019], and ICM [Pathak et al., 2017] augment a trained policy's reward to seek novelty; they need 10^6–10^8 steps to warm up, so at a tester's budget the bonus is cold. *We match the budget and train nothing.* **Go-Explore / archive methods.** Go-Explore [Ecoffet et al., 2021] keeps an archive of returnable states and is the strongest classical family, but its wins hinge on a hand-chosen cell abstraction and are shown per game. *We show a plain archive is not enough, since the win is the discovered strategy over it, and evaluate across engines.* **Action-repeat.** A stochasticity control [Machado et al., 2018], not an exploration method. *We include it as a baseline; it is our tightest competitor on physics and our weakest on navigation.* **LLMs that write code for control.** FunSearch [Romera-Paredes et al., 2024] evolves programs; Eureka [Ma et al., 2024] writes rewards and trains RL; PromptPO [Hatgis-Kessell and Brunskill, 2026] writes control policies for return; TITAN [Wang et al., 2025] runs a per-step LLM agent as a tester. *We occupy the point they leave open: budget-matched, target-reaching, and replay-verified, a written exploration program that is paid once and run training-free.* Table 1 places the families on the axes that matter.

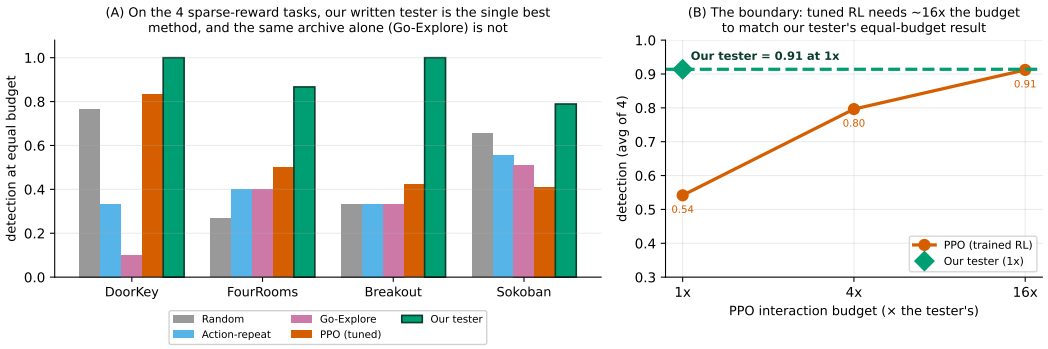


Figure 1: **(A)** At an equal interaction budget on the four sparse-reward tasks, our tester is the single best method, above random, action-repeat, plain Go-Explore (the *same* save/restore archive), and tuned PPO. Go-Explore's 0.10 on DoorKey shows the win is the discovered strategy, not the archive. **(B)** The boundary: only PPO's budget scales, and tuned PPO needs about $16\times$ the interaction to match our tester's $1\times$ result (PPO is best-of-3 objectives at $4/16\times$).

1.3 The result

The equal-budget win (Fig. 1A). We report one program, unchanged across tasks, reaching on average 0.91 of the targets against 0.54 for tuned PPO. Per task it beats every baseline at equal budget: DoorKey 1.00, FourRooms 0.87, Breakout 1.00, Sokoban 0.79, against tuned PPO $0.83/0.50/0.42/0.41$, action-repeat $0.33/0.40/0.33/0.56$, Go-Explore $0.10/0.40/0.33/0.51$, and random $0.77/0.27/0.33/0.66$. These are the sparse-reward, non-saturated tasks in the suite, chosen along one axis, reward density; on them, winning requires real exploration that no cheap baseline provides. Our tester's full record on every task, including where it ties or loses, is the map below.

The boundary is the thesis (Fig. 1B). The claim is strictly equal-budget. Scaling only PPO's budget, its average climbs from 0.54 at $1\times$ to 0.91 at $16\times$, matching what our tester reaches at $1\times$; PPO overtakes our tester on DoorKey at $4\times$ and on Breakout and Sokoban at $16\times$. We explicitly withdraw any "RL never catches up" reading: it does, given an order of magnitude more interaction. That split is the contribution. Write once and win at the tester's budget, or train and win with a larger one.

Strategy, not archive; and it transfers. Plain Go-Explore has the *same* archive yet scores 0.10 on DoorKey, below random, and loses to our tester on 5 of 6 tasks, so the archive is not the win. And the win transfers across engines: our tester was evolved once on a multi-engine fitness (Kinetix, Craftax, and Gymnax) that never saw MiniGrid, MinAtar, or Sokoban, while a *fresh* DoorKey-only search reaches just 0.33 (beating Go-Explore, losing to transfer). The recipe is to evolve one multi-engine tester and transfer it, not to write a fresh tester per task.

The map (where a written program does *not* win). Our tester's full row: on MountainCar and DeepSea-24 it scores 1.00, but so does trivial action-repeat, so these are ceiling ties with no exploration skill on display (excluded from the claim above); on reward-dense **Craftax it loses**, 0.27 against PPO 0.34; and it does not run at Kinetix protocol scale, a compile-time limit of its archive that sibling programs cover (Section 1.4). This is the regime boundary: writing wins where reward is sparse and the budget is tight; on our one reward-dense task and at 4–$16\times$ the budget, training wins.

1.4 Setup and the verification backbone

Testers as programs (Fig. 2). A tester is four pure-JAX hooks (`init/restore/act/archive`) over an archive of save/restore-able states, run as B workers for T steps, so the budget is $B{\cdot}T$; the same interface runs on every engine through a generic feature head. PPO is trained on the *same* feature head, with no observation handicap, and its training steps *are* its budget. The equal-budget PPO column (Fig. 1A) is best over a 5-objective sweep (count, RND, ICM, proximity-curiosity, and task reward) crossed with {default, tuned $lr{=}10^{-4}$, ent$=$0.2}; the crossover curves (Fig. 1B) are best-of-3 objectives, labeled as such. Testers use 5 seeds and PPO 3 to 5; we report one fixed program.

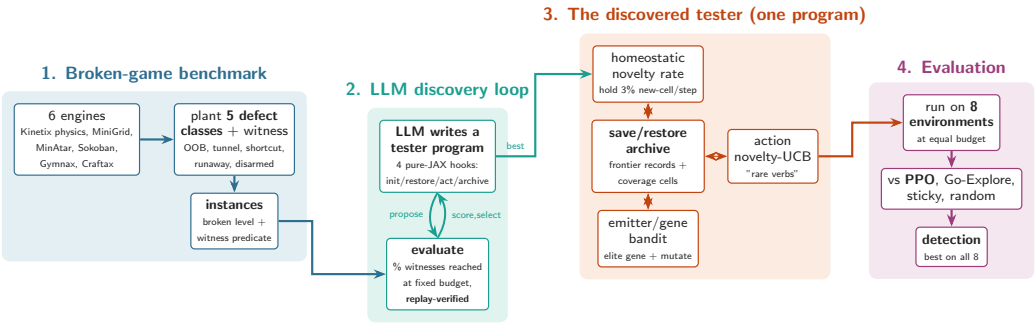


Figure 2: The loop: break six engines into defect classes with programmatic witnesses; an LLM writes and mutates tester programs scored on a multi-engine fitness (Kinetix hits replayed from reset, Section 1.4); the single champion program explores through a save/restore archive driven by three runtime modules; evaluate against PPO, Go-Explore, action-repeat, and random at equal budget.

Depth and verification (Kinetix planted defects). The rigor lives here. We plant five defect classes (OOB, TUNNEL, i.e. clipping, SHORTCUT, ANOMALY, DISARMED) with programmatic witness predicates over 74 shipped and 64 generated levels, and **every counted hit is re-simulated from a clean level reset and must re-raise the witness, or it is a miss**. The protocol is frozen and hashed; a 30% level split and a whole defect class are sealed from the search, and a reachability calibration bounds what any tester can witness. The loop's strongest Kinetix program (a sibling of our tester, which does not run at this scale) leads a large method field on the search split (0.655 against action-repeat 0.602, PPO 0.566, random 0.494), but that is *in-sample* selection, and on held-out levels its lead over action-repeat narrows to a near-tie, so we headline the apparatus rather than the search-split margin. Section 2.5 recounts how this apparatus caught the loop inflating its own fitness by $+0.11$ through a harness race, the check the whole result rests on.

2 System Design and Meta-Analysis

This Part is the human-authored companion to the result above: a meta-analysis of how the work was actually produced, by which agents, and how its claims were checked.

2.1 Agent and Harness

Our main loop is an island program-evolution framework (ShinkaEvolve [Lange et al., 2025]) with two Claude models as the proposal agents that wrote and mutated candidate tester programs. We use `claude-sonnet-5` and `claude-opus-5`. We ran two islands under a fixed $40 API-cost cap (the run spent about $20), and the champion emerged after roughly 35 generations. This is the loop which evolved testers, ran experiments, and drafted Part 1.

For the model that drove the project, we used `claude-fable-5`. To begin the project, we wrote a large, detailed outline of the research direction and had the agent pursue it autonomously.

The harness. Around the proposal models, ShinkaEvolve runs a fixed three-stage cycle. In *proposal*, a model is given the current champion, the task description, the four-hook program interface, and the scoring rule, and returns an edited candidate program. In *evaluation*, that candidate is compiled and executed on every substrate at a fixed interaction budget, and scored by the fraction of witness / target states it reaches. In *selection*, candidates are kept in a two-island archive with a bandit over the two proposal models, and the best-scoring program is carried forward. A tester itself is four pure-JAX hooks (`init`, `restore`, `act`, `archive`) over a save/restore archive of simulator states, run as B parallel workers for T steps (an interaction budget of $B \cdot T$), behind a generic feature head so the same program runs unchanged on every engine.

2.2 Tools

Models and framework. The proposal models (`claude-sonnet-5`, `claude-opus-5`) were accessed through a headless Claude API and orchestrated by ShinkaEvolve [Lange et al., 2025]; the driver model was `claude-fable-5`. **Substrates.** We use genuine published simulators, which are all pure-JAX (Kinetix/jax2d, MiniGrid, MinAtar, Jumanji Sokoban, Gymnax, Craftax). The PPO, Go-Explore, action-repeat, and random baselines run in the same harness on the same feature vector. **Fitness and compute.** Candidates were scored by the fraction of witness states reached at a fixed budget, on a multi-engine search split (seeds $\{0, 1, 2\}$), run across an Apple M4 Pro laptop and 4 NVIDIA A40 GPUs. **Feedback.** Each candidate received back its scalar fitness together with its worst-scoring targets and a paired 95% confidence interval against a frozen reference program.

2.3 Research Loop

The loop repeated the proposal $\rightarrow$ evaluate $\rightarrow$ select cycle above. Each hypothesis took the form of a program edit, a new novelty signal, a bandit over exploration moves, a different archive policy, and was tested the only way the harness allows: by executing the edited program on the multi-engine fitness and keeping or discarding it by score. Analysis, comparing the champion against PPO, Go-Explore, action-repeat, and random at equal budget and on held-out engines, ran in the same harness. The run completed 37 generations (it began on 2026-08-29) and recorded 38 candidate programs; the champion appeared at generation 35, at a total API cost of about $20.

2.4 Human Interventions

Human involvement was steering, not execution. Throughout the process, we continuously asked for reality-checking results, and ran multiple adversarial reviews. On average, we checked in twice a day to confirm research direction and delegate new tasks for the loop to complete. Multiple re-frames of the research vision were required, as sometimes the loop would tunnel vision into claims which were not useful. Whenever the agent was asked to do an adversarial review, it would overcorrect and severely downplay many core findings, and so additional input from us was required to correct that.

2.5 Verification

We found verification is required. The loop's first strong discovery inflated its own fitness by $+0.11$ through a concurrency race in the harness (reaching physically impossible states, with nothing wrong in the candidate's code). It passed the fitness, both held-out splits, four design comparisons, and an LLM code-audit. Only when we replayed every hit from a clean reset did we catch the bug. We rebuilt the harness, withdrew the number, and corrected our own $N{=}3$ over-reach to $N{=}1$ (three claimed wins down to the one that survived replay). This shows that when we are optimizing for a score with an autonomous research loop, it is especially important to ensure that the harness and testing environments are robust and correct.

Concretely, four mechanisms carry the verification. (1) *Replay-from-reset*: every counted Kinetix hit is re-simulated from a clean level reset and must re-raise its witness predicate, or it is discarded, this is the check that caught the exploit. (2) A *frozen, hashed protocol* fixing splits, seeds, budgets, and exclusions, so no comparison can drift between runs. (3) *Sealed held-out data*: a 30% level split and an entire defect class the search never sees. (4) A *reachability calibration* that bounds what any tester could witness, so a score is read against what is actually attainable rather than against a nominal 100%.

2.6 Significance of result

We are able to reach higher capabilities given the same cost. A written tester is a program that runs *training-free*. It is only necessary to evolve one transferable tester once, then reuse it across games, versus training a fresh RL explorer for each. At a tester's real budget the written program wins, and RL only catches up given far more compute, so for the same testing outcome the written route is much cheaper to deploy. In this work, that one-time cost was about $20 of API calls.

Our approach is far more accessible. Having cheaper game testing agents lowers the bar for who can test a game. Given the accessibility of coding agents, it is very feasible that a small studio or an

individual without access to enough compute for an RL-training pipeline but with access to an LLM can now run a written tester to explore their game for bugs. Having cheap bug tests allows game developers to focus more on pushing features rather than searching for complex bugs.

2.7 Interpretability

Every artifact of the search is preserved, so the process can be audited end to end. ShinkaEvolve records each candidate program in a database keyed by generation and island, with its fitness and its lineage; each generation's candidate source is also written to disk, and the champion is a single, human-readable pure-JAX file. The evaluation protocol is frozen and content-hashed, and every reported number regenerates from logs by re-running the fixed splits and seeds; result tables are produced by scripts from those logs rather than transcribed by hand. Runs, code, and the benchmark are under version control. This is exactly what let us reconstruct, after the fact, which generation produced the champion, at what cost, and that an earlier, better-looking "discovery" was a harness artifact rather than a real result.

2.8 Limitations

The claim is equal-budget on sparse-reward target-reaching. If we provide 4–$16\times$ more interaction, tuned PPO catches up (Fig. 1B). Action-repeat is even cheaper than our method, and is a close competitor on Kinetix *physics*. Our tester is also not universal. It ties two saturated tasks a trivial baseline also solves, loses reward-dense Craftax, and does not run at Kinetix protocol scale. It is also important to note that many of the games are very simple, and all are in 2D space. Some are not even games; Kinetix is a research substrate. In order to help this generalize further into real games with complex horizons and loops, much more work is required.

3 Broader Impact

The impacts of this project for game development should generally be positive. The goal was to make autonomous testing more accessible, and with the widespread use of large language models in the current world, we believe many people can adopt and utilize this work. Though additional work would be necessary to have it work on complex games.

Throughout the process of this autonomous loop, we found that we needed to intervene quite often. The agent had misreported its correctness at times. At one point, the harness it wrote was broken, which inflated its score. It required a lot of double-checking results in order to find the leak. It is interesting to think about why it took a while; maybe it was a hard problem, but also maybe the agent was poised towards having a greater efficacy, and so it did not wish to look for such a result. Only when it was told that it was its top priority to report absolutely concrete and correct findings did it seem to look. When doing autonomous research, if your metric for success is something like accuracy, it is integral to build checks to ensure the results are correct, since the large language model seems to lack surrounding sight.

The agent would also consistently overcorrect. If we mentioned that the direction seemed a bit vague, it would tunnel vision immediately. If we ran an adversarial review and nitpicked at its work, then it would overcorrect. A theory we have is that large language models tend to over-exaggerate most things, because users enjoy this, and so it takes criticism and overcorrects to it in the same over-exaggeratory manner.

It is clear that autonomous research requires a strong research direction, with a strong harness to keep it there. It needs something which is able to find the nuance between correcting to a director's opinion and fulfilling the research vision. It is unclear if autonomous research can come up with its own grander ideas, as it seemed that throughout this project, we needed to provide it with its next steps quite often. An example of lack of creativity is its continuous refining of the same strategy rather than doing research into possibly better architectures. The current finding was only possible due to telling the model to explore and draft up separate proposals.

Autonomous research sits in a strange place. It calls for a shift in how people approach research. Previously, you needed to be an expert in a field just to take a step in that direction. Now, especially in software with coding agents, you do not need to be such an expert. You simply must steer the

wheel of an (seemingly) all-knowing ship that will complete the tasks you specify. As we spoke about in the last paragraph, this ship seems to lack much creativity and direction, and so researchers are absolutely still necessary. But it is extremely exciting. Autonomous coding agents can contribute a thousandfold more to open source projects and research. People who have good ideas and interesting perspectives but were not well versed in a niche can contribute. The goal of research seems to be to advance science, after all.

A Reproducibility Statement

The primary result is the equal-budget comparison in Fig. 1 and the Kinetix depth study in Section 1.4. Reproducing it requires: the champion tester program (a single pure-JAX file) and the sibling programs; the frozen, content-hashed protocol (level and defect-class splits, seeds 100–104 for evaluation and 0–2 for fitness, budgets, and exclusions); and the baselines (PPO, Go-Explore, action-repeat, random) run in the same harness on the same feature head. Compute: orchestration on an Apple M4 Pro laptop and evaluation on 4 NVIDIA A40 GPUs; the evolutionary search used ShinkaEvolve with two islands and `claude-sonnet-5` / `claude-opus-5` as proposal models, completing 37 generations at about $20 in API cost. Every counted Kinetix hit is verified by replay-from-reset (Section 2.5): reproduction re-runs the fixed protocol and requires each hit to re-raise its witness predicate from a clean reset, which is what distinguishes a real detection from a harness artifact.

B Disclosure Statement

Agent(s) / model(s) used. The tester programs and Part 1 were produced by an autonomous ShinkaEvolve loop using `claude-sonnet-5` and `claude-opus-5` as proposal models. The project was driven by an interactive Claude agent (`claude-fable-5`) used for planning, analysis, reality-checking, adversarial review, and drafting. Part 2, Part 3, and this statement are human-authored. Every claim, figure, and number in Part 1 was verified by the human author against regenerable logs and the replay verifier described in Section 2.5.

- ☐ A human author has curated this work and verified every claim, figure, methodology detail, and reference.
- ☐ A human author takes full accountability for the submission as a human-authored, human-verified artifact.

References

Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations (ICLR)*, 2019. URL `https://arxiv.org/abs/1810.12894`.

Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. First return, then explore. *Nature*, 590:580–586, 2021. doi: 10.1038/s41586-020-03157-9.

Stephane Hatgis-Kessell and Emma Brunskill. When are LLMs sufficient policy optimizers for sequential RL tasks? *arXiv preprint arXiv:2605.30719*, 2026.

Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. ShinkaEvolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*, 2025.

Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

Marlos C. Machado, Marc G. Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018. doi: 10.1613/jair.5699.

Deepak Pathak, Pulkit Agrawal, Alexei A. Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024. doi: 10.1038/s41586-023-06924-6.

Haoran Tang, Rein Houthooft, Davis Foote, Adam Stooke, Xi Chen, Yan Duan, John Schulman, Filip De Turck, and Pieter Abbeel. #exploration: A study of count-based exploration for deep reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. URL https://arxiv.org/abs/1611.04717.

Chengjia Wang, Lanling Tang, Ming Yuan, Jiongchi Yu, Xiaofei Xie, and Jiajun Bu. Leveraging llm agents for automated video game testing. *arXiv preprint arXiv:2509.22170*, 2025.